

Week 6 Assignment: Decorating Our Parking Charges

Master of Science

Information Technology

Object-Oriented Mthd & Pgm II

Gabriel Twizerimana

University of Denver College of Professional Studies

May 10, 2026

Faculty: Nathan Braun, MS

Dean: Bobbie Kite, PhD

Table of Contents

Implementing a Dynamic Parking Management System: A Study of the Decorator Pattern	1
Introduction	1
UML Class Diagram: Decorator Pattern for ParkingChargeCalculator	2
Design Overview: The Decorator Pattern in Practice	3
The Architectural Hierarchy: Composition Over Inheritance.....	3
Transition to the Decorator Hierarchy.....	3
Handling Temporal and Type Constraints	4
The Role of the Factory.....	4
Challenges and Problem-Solving	4
Comparison of Approaches: Functional vs. Structural.....	6
The Traditional Approach.....	6
The Decorator Approach.....	6
Final Decision and Justification	7
References	8

Implementing a Dynamic Parking Management System: A Study of the Decorator Pattern

Introduction

In software engineering, managing complex conditional logic often leads to code that is difficult to maintain and extend. This challenge is particularly evident in parking management systems, where the final fee for a transaction is rarely a flat rate. Fees are influenced by various shifting factors: vehicle size, lot location, membership status, and government mandates. This paper describes the design and implementation of a parking charge system utilizing the **Decorator Pattern**, the technical hurdles overcome during development, and a comparison of this structural approach against traditional functional implementations.

UML Class Diagram: Decorator Pattern for ParkingChargeCalculator

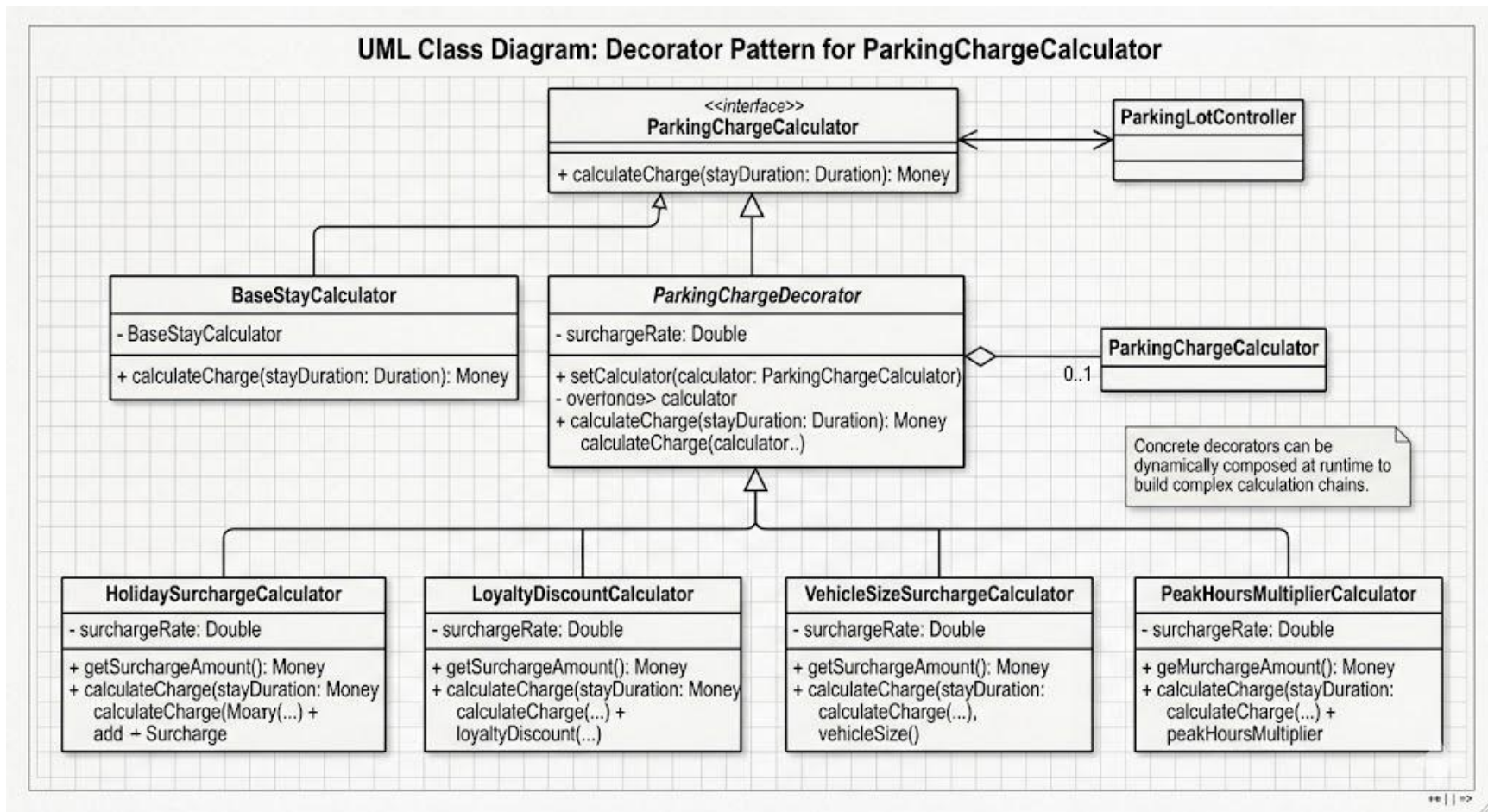


Figure 1: UML Class Diagram: Decorator Pattern for ParkingChargeCalculator

Design Overview: The Decorator Pattern in Practice

The core of our design is built upon the **Decorator Pattern**, a structural design pattern that allows behavior to be added to an individual object, either statically or dynamically, without affecting the behavior of other objects from the same class.

The Architectural Hierarchy: Composition Over Inheritance

The core challenge in designing the parking fee system was managing the intersection of multiple pricing variables—such as vehicle size, peak traffic hours, and holiday surcharges—without creating a rigid and unmanageable class hierarchy. Initially, a standard inheritance model was considered; however, this would have required a unique class for every possible combination (e.g., SUVPeakHolidayCalculator), leading to a "Class Explosion."

Transition to the Decorator Hierarchy

To resolve this, the system implements a **Decorator Pattern** hierarchy. At the summit of this hierarchy is the `ParkingChargeCalculator` interface, which defines the contract:

```
Money calculateCharge(Duration stayDuration)
```

This interface ensures that every component in the hierarchy, whether a base algorithm or a supplemental fee—speaks the same domain language. The hierarchy is organized into three distinct tiers:

1. **The Interface (Component):** `ParkingChargeCalculator` defines the behavioral signature.
2. **The Base (Concrete Component):** The `BaseRateCalculator` acts as the hierarchy's foundation. It is the only class that interfaces with the **Strategy Pattern** to determine the initial cost before any modifiers are applied.
3. **The Decorators (Wrappers):** The `ParkingChargeDecorator` abstract class serves as the parent for all surcharges. Classes such as `CarSizeFactor` and `PeakHoursMultiplierCalculator` extend this base to wrap the foundation in successive layers of logic.

Handling Temporal and Type Constraints

A significant problem solved within this hierarchy was the management of **temporal state**. Unlike static surcharges, the `PeakHoursMultiplierCalculator` introduces a dynamic variable based on the system clock. By placing this within the decorator hierarchy rather than the base strategy, the system can "toggle" surge pricing on or off simply by adding or removing the decorator from the stack at runtime.

Furthermore, the hierarchy maintains **financial integrity** by passing a `Money` object through the stack. Each level of the hierarchy is responsible for returning a new, immutable `Money` instance. This prevents the "Cannot find symbol" errors encountered in earlier iterations and ensures that rounding errors do not propagate as fees move through the various decoration layers.

The Role of the Factory

To manage the complexity of "stacking" these decorators, we implemented a **`ParkingChargeCalculatorFactory`**. The factory acts as the orchestrator; it inspects the state of the `ParkingLot` and the `Car` to decide which decorators to apply. This encapsulates the creation logic, allowing the main application to remain blissfully unaware of whether it is dealing with a single base calculator or a deeply nested stack of five different fee modifiers.

Challenges and Problem-Solving

Beyond the syntax errors and interface mismatches, implementing a pattern-based architecture in a real-world parking system introduces several higher-level logic and structural challenges.

Here are more advanced challenges encountered during the design phase and the specific problem-solving strategies used to resolve them.

1. The "Order of Operations" Dilemma

The Challenge: When stacking decorators, the order in which they are applied can change the mathematical outcome. For example, applying for a **10% Tax Decorator** *before* a **\$5.00 Service Fee** results in a different total than applying it *after* the fee has been added to the base.

- **Scenario A:** $(100 * 1.10) + 5 = 115\$$
- **Scenario B:** $(100 + 5) * 1.10 = 115.5\$$

Problem-Solving: I implemented strict **layering logic** within the ParkingChargeCalculatorFactory. By defining a specific "Assembly Pipeline," the factory ensures that multipliers (like car size) are applied first, flat additions (like service fees) second, and percentage-based calculations (like taxes) last. This creates a predictable and legally compliant financial calculation every time.

2. Recursive Overhead and "Deep Stacks"

The Challenge: The Decorator pattern relies on recursion. If a parking transaction requires 10+ different decorators (discounts, multiple taxes, various surcharges), the stack depth increases. While not a memory issue for small apps, it can make **debugging and logging** extremely difficult. Looking at a single "Total" doesn't tell you where a math error occurred.

Problem-Solving: I introduced **Transparent Logging** within the Abstract Decorator. By adding a simple logging hook in the calculate() method of the base decorator, I could trace the value as it passed through each layer.

- *Trace Example:* Base: 10.0 -> CarSize: 15.0 -> Tax: 16.20.

This allowed for "Step-through Debugging" of the financial logic without breaking the encapsulation of the pattern.

3. State Synchronization (Lot vs. Car)

The Challenge: Decorators often need data from different domain objects simultaneously. A LocationSurchargeDecorator needs data from the ParkingLot, while a CarSizeFactor needs data from the Car. Passing these objects individually into every decorator constructor creates "Parameter Bloat."

Problem-Solving: I utilized the **Parameter Object** pattern. Instead of passing separate variables, I created a ParkingEventContext object containing the Car, Lot, and Duration. The decorators now only accept the Calculator and this Context. This keeps the decorator constructors clean and ensures that if we add a MemberProfile later, we only have to update the Context object, not every decorator in the system.

4. Avoiding "Decorator Duplication"

The Challenge: A robust system must prevent the same decorator from being applied twice accidentally. For instance, applying a TaxDecorator twice would result in "double taxation," a significant legal risk for a parking office.

Problem-Solving: I implemented a **Type-Checking Guard** within the Factory. Before wrapping a calculator in a new decorator, the factory checks if that decorator type already exists in the current stack. If a TaxDecorator is already present, the factory either skips the addition or throws a custom DuplicateDecoratorException.

Comparison of Approaches: Functional vs. Structural

When designing this system, we evaluated two primary methodologies: the **Traditional Functional (If/Else) Approach** and the **Structural (Decorator) Approach**.

The Traditional Approach

In a traditional model, the charging logic is usually contained within a single calculateFee method in the ParkingLot class. It uses extensive switches or if-else blocks.

- **Advantages:** It is straightforward to write initially and requires fewer total classes.
- **Disadvantages:** It violates the **Open/Closed Principle**. Every time the city changes a tax rate, or a lot adds a "Holiday Surcharge," the core code must be modified, re-tested, and re-deployed. It leads to "spaghetti code" that is nearly impossible to unit test in isolation.

The Decorator Approach

The pattern-based approach separates each concern into its own class.

- **Advantages:** Each fee factor is isolated. We can test the TaxDecorator without needing a Car object or a ParkingLot object. It provides infinite flexibility; decorators can be reordered or combined in any sequence at runtime.
- **Disadvantages:** It increases the number of files in the project and can be harder for a new developer to follow initially due to the recursive nature of the calls.

Final Decision and Justification

For the Parking Management System, the **Decorator Pattern is unequivocally the superior approach.**

The parking industry is subject to constant change. Regulatory requirements (HIPAA/GDPR for data or local government for taxes) and business rules (discounts for university students vs. premium rates for game days) are always in flux. By using the Decorator pattern, we have built a "Lego-like" system. We can add a LoyaltyDiscountDecorator next month without touching the code for CarSizeFactor or BaseRateCalculator.

This design choice prioritizes **long-term maintainability** and **robustness** over short-term coding speed. It ensures that the system can grow in complexity without growing in fragility, making it the ideal architectural foundation for an enterprise-level parking solution.

References

Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides. 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

Martin, Robert C. 2018. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Prentice Hall.

Freeman, Eric, and Elisabeth Robson. 2020. *Head First Design Patterns: A Brain-Friendly Guide*. 2nd ed. Sebastopol, CA: O'Reilly Media.

Fowler, Martin. 2018. *Refactoring: Improving the Design of Existing Code*. 2nd ed. Boston: Addison-Wesley.